```
StorageFolder library = Windows.Storage.KnownFolders.PicturesLibrary;
IReadOnlyList<StorageFile> files = await
library.GetFilesAsync(Windows.Storage.Search.CommonFileQuery.OrderByDate);
for (int i = 0; i < files.Count; i++)
{
    ImageProperties imgProps = await
files[i].Properties.GetImagePropertiesAsync();

    // do something with the date the image was taken
    DateTimeOffset date = imgProps.DateTaken;
}
```

VB

```
Dim library As StorageFolder = Windows.Storage.KnownFolders.PicturesLibrary
Dim files As IReadOnlyList(Of StorageFile) = Await
library.GetFilesAsync(Windows.Storage.Search.CommonFileQuery.OrderByDate)
For i As Integer = 0 To files.Count - 1
    Dim imgProps As ImageProperties =
        Await files(i).Properties.GetImagePropertiesAsync()

    ' do something with the date the image was taken
    Dim dateTaken As DateTimeOffset = imgProps.DateTaken
Next i
```

The third example uses **QueryOptions** to get info about a set of files. This approach provides much better performance than the previous example.

C#

```
// Set QueryOptions to prefetch our specific properties
var queryOptions = new
Windows.Storage.Search.QueryOptions(CommonFileQuery.OrderByDate, null);
queryOptions.SetThumbnailPrefetch(ThumbnailMode.PicturesView, 100,
        ThumbnailOptions.ReturnOnlyIfCached);
queryOptions.SetPropertyPrefetch(PropertyPrefetchOptions.ImageProperties,
       new string[] {"System.Size"});

StorageFileQueryResult queryResults =
KnownFolders.PicturesLibrary.CreateFileQueryWithOptions(queryOptions);
IReadOnlyList<StorageFile> files = await queryResults.GetFilesAsync();

foreach (var file in files)
{
    ImageProperties imageProperties = await
file.Properties.GetImagePropertiesAsync();

    // Do something with the date the image was taken.
    DateTimeOffset dateTaken = imageProperties.DateTaken;

    // Performance gains increase with the number of properties that are
```

```
accessed.
    IDictionary<String, object> propertyResults =
        await file.Properties.RetrievePropertiesAsync(
              new string[] {"System.Size" });

    // Get/Set extra properties here
    var systemSize = propertyResults["System.Size"];
}
```

VB

```
' Set QueryOptions to prefetch our specific properties
Dim queryOptions = New
Windows.Storage.Search.QueryOptions(CommonFileQuery.OrderByDate, Nothing)
queryOptions.SetThumbnailPrefetch(ThumbnailMode.PicturesView,
            100,
Windows.Storage.FileProperties.ThumbnailOptions.ReturnOnlyIfCached)
queryOptions.SetPropertyPrefetch(PropertyPrefetchOptions.ImageProperties,
                                 New String() {"System.Size"})

Dim queryResults As StorageFileQueryResult =
KnownFolders.PicturesLibrary.CreateFileQueryWithOptions(queryOptions)
Dim files As IReadOnlyList(Of StorageFile) = Await
queryResults.GetFilesAsync()


For Each file In files
    Dim imageProperties As ImageProperties = Await
file.Properties.GetImagePropertiesAsync()

    ' Do something with the date the image was taken.
    Dim dateTaken As DateTimeOffset = imageProperties.DateTaken

    ' Performance gains increase with the number of properties that are
accessed.
    Dim propertyResults As IDictionary(Of String, Object) =
        Await file.Properties.RetrievePropertiesAsync(New String()
{"System.Size"})

    ' Get/Set extra properties here
    Dim systemSize = propertyResults("System.Size")

Next file
```

If you're performing multiple operations on Windows.Storage objects such as `Windows.Storage.ApplicationData.Current.LocalFolder`, create a local variable to reference that storage source so that you don't recreate intermediate objects each time you access it.

Stream performance in C# and Visual Basic

Buffering between UWP and .NET streams

There are many scenarios when you might want to convert a UWP stream (such as a Windows.Storage.Streams.IInputStream or IOutputStream) to a .NET stream (System.IO.Stream). For example, this is useful when you are writing a UWP app and want to use existing .NET code that works on streams with the UWP file system. In order to enable this, .NET APIs for UWP apps provides extension methods that allow you to convert between .NET and UWP stream types. For more info, see WindowsRuntimeStreamExtensions.

When you convert a UWP stream to a .NET stream, you effectively create an adapter for the underlying UWP stream. Under some circumstances, there is a runtime cost associated with invoking methods on UWP streams. This may affect the speed of your app, especially in scenarios where you perform many small, frequent read or write operations.

In order to speed up apps, the UWP stream adapters contain a data buffer. The following code sample demonstrates small consecutive reads using a UWP stream adapter with a default buffer size.

C#

```
StorageFile file = await Windows.Storage.ApplicationData.Current
    .LocalFolder.GetFileAsync("example.txt");
Windows.Storage.Streams.IInputStream windowsRuntimeStream =
    await file.OpenReadAsync();

byte[] destinationArray = new byte[8];

// Create an adapter with the default buffer size.
using (var managedStream = windowsRuntimeStream.AsStreamForRead())
{

    // Read 8 bytes into destinationArray.
    // A larger block is actually read from the underlying
    // windowsRuntimeStream and buffered within the adapter.
    await managedStream.ReadAsync(destinationArray, 0, 8);

    // Read 8 more bytes into destinationArray.
    // This call may complete much faster than the first call
    // because the data is buffered and no call to the
    // underlying windowsRuntimeStream needs to be made.
    await managedStream.ReadAsync(destinationArray, 0, 8);
}
```

VB

```
Dim file As StorageFile = Await Windows.Storage.ApplicationData.Current -
.LocalFolder.GetFileAsync("example.txt")
Dim windowsRuntimeStream As Windows.Storage.Streams.IInputStream =
    Await file.OpenReadAsync()

Dim destinationArray() As Byte = New Byte(8) {}

' Create an adapter with the default buffer size.
Dim managedStream As Stream = windowsRuntimeStream.AsStreamForRead()
Using (managedStream)

    ' Read 8 bytes into destinationArray.
    ' A larger block is actually read from the underlying
    ' windowsRuntimeStream and buffered within the adapter.
    Await managedStream.ReadAsync(destinationArray, 0, 8)

    ' Read 8 more bytes into destinationArray.
    ' This call may complete much faster than the first call
    ' because the data is buffered and no call to the
    ' underlying windowsRuntimeStream needs to be made.
    Await managedStream.ReadAsync(destinationArray, 0, 8)

End Using
```

This default buffering behavior is desirable in most scenarios where you convert a UWP stream to a .NET stream. However, in some scenarios you may want to tweak the buffering behavior in order to increase performance.

Working with large data sets

When reading or writing larger sets of data you may be able to increase your read or write throughput by providing a large buffer size to the **AsStreamForRead**, **AsStreamForWrite**, and **AsStream** extension methods. This gives the stream adapter a larger internal buffer size. For instance, when passing a stream that comes from a large file to an XML parser, the parser can make many sequential small reads from the stream. A large buffer can reduce the number of calls to the underlying UWP stream and boost performance.

> **Note** You should be careful when setting a buffer size that is larger than approximately 80 KB, as this may cause fragmentation on the garbage collector heap (see Improve garbage collection performance). The following code example creates a managed stream adapter with an 81,920 byte buffer.

C#

```
// Create a stream adapter with an 80 KB buffer.
Stream managedStream = nativeStream.AsStreamForRead(bufferSize: 81920);
```

VB

```
' Create a stream adapter with an 80 KB buffer.
Dim managedStream As Stream =
nativeStream.AsStreamForRead(bufferSize:=81920)
```

The Stream.CopyTo and CopyToAsync methods also allocate a local buffer for copying between streams. As with the AsStreamForRead extension method, you may be able to get better performance for large stream copies by overriding the default buffer size. The following code example demonstrates changing the default buffer size of a **CopyToAsync** call.

C#

```
MemoryStream destination = new MemoryStream();
// copies the buffer into memory using the default copy buffer
await managedStream.CopyToAsync(destination);

// Copy the buffer into memory using a 1 MB copy buffer.
await managedStream.CopyToAsync(destination, bufferSize: 1024 * 1024);
```

VB

```
Dim destination As MemoryStream = New MemoryStream()
' copies the buffer into memory using the default copy buffer
Await managedStream.CopyToAsync(destination)

' Copy the buffer into memory using a 1 MB copy buffer.
Await managedStream.CopyToAsync(destination, bufferSize:=1024 * 1024)
```

This example uses a buffer size of 1 MB, which is greater than the 80 KB previously recommended. Using such a large buffer can improve throughput of the copy operation for very large data sets (that is, several hundred megabytes). However, this buffer is allocated on the large object heap and could potentially degrade garbage collection performance. You should only use large buffer sizes if it will noticeably improve the performance of your app.

When you are working with a large number of streams simultaneously, you might want to reduce or eliminate the memory overhead of the buffer. You can specify a smaller buffer, or set the *bufferSize* parameter to 0 to turn off buffering entirely for that stream

adapter. You can still achieve good throughput performance without buffering if you perform large reads and writes to the managed stream.

Performing latency-sensitive operations

You might also want to avoid buffering if you want low-latency reads and writes and do not want to read in large blocks out of the underlying UWP stream. For example, you might want low-latency reads and writes if you are using the stream for network communications.

In a chat app you might use a stream over a network interface to send messages back and forth. In this case you want to send messages as soon as they are ready and not wait for the buffer to fill up. If you set the buffer size to 0 when calling the **AsStreamForRead**, **AsStreamForWrite**, and **AsStream** extension methods, then the resulting adapter will not allocate a buffer, and all calls will manipulate the underlying UWP stream directly.

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Windows Runtime Components and optimizing interop

Article • 10/20/2022

Create Windows apps that use Windows Runtime Components and interop between native and managed types while avoiding interop performance issues.

Best practices for interoperability with Windows Runtime Components

If you are not careful, using Windows Runtime Components can have a large impact on your app performance. This section discusses how to get good performance when your app uses Windows Runtime Components.

Introduction

Interoperability can have a big impact on performance and you might be using it without even realizing that you are. The Windows Runtime handles a lot of the interoperability for you so that you can be more productive and reuse code that was written in other languages. We encourage you to take advantage of what the Windows Runtime does for you, but be aware that it can impact performance. This section discusses things you can do to lessen the impact that interoperability has on your app's performance.

The Windows Runtime has a library of types that are accessible from any language that can write a Universal Windows Platform app. You use the Windows Runtime types in C# or Microsoft Visual Basic the same way you use .NET objects. You don't need to make platform invoke method calls to access the Windows Runtime components. This makes writing your apps much less complex, but it is important to realize that there might be more interoperability occurring than you expect. If a Windows Runtime component is written in a language other than C# or Visual Basic, you cross interoperability boundaries when you use that component. Crossing interoperability boundaries can impact the performance of an app.

When you develop a Universal Windows Platform app in C# or Visual Basic, the two most common set of APIs that you use are the Windows Runtime APIs and the .NET APIs for UWP apps. In general, types provided by Windows that are built on the Windows Runtime in namespaces that begin with "Windows." and .NET types are in namespaces that begin with "System." There are exceptions, though. The types in .NET for UWP apps

do not require interoperability when they are used. If you find that you have bad performance in an area that uses then Windows Runtime, you might be able to use .NET for UWP apps instead to get better performance.

Note Most of the Windows Runtime components that ship with Windows 10 are implemented in C++ so you cross interoperability boundaries when you use them from C# or Visual Basic. As always, make sure to measure your app to see if using Windows Runtime components affects your app's performance before you invest in making changes to your code.

In this topic, when we mention "Windows Runtime components", we mean Windows Runtime Components that are written in a language other than C# or Visual Basic.

Each time you access a property or call a method on a Windows Runtime component, an interoperability cost is incurred. In fact, creating a Windows Runtime component is more costly than creating a .NET object. The reasons for this are that the Windows Runtime must execute code that transitions from your app's language to the component's language. Also, if you pass data to the component, the data must be converted between managed and unmanaged types.

Using Windows Runtime Components efficiently

If you find that you need to get better performance, you can ensure that your code uses Windows Runtime components as efficiently as possible. This section discusses some tips for improving performance when you use Windows Runtime components.

It takes a significant number of calls in a short period of time for the performance impact to be noticeable. A well-designed application that encapsulates calls to Windows Runtime components from business logic and other managed code should not incur huge interoperability costs. But if your tests indicate that using Windows Runtime components is affecting your app's performance, the tips discussed in this section help you improve performance.

Consider using types provided by .NET for UWP apps

There are certain cases where you can accomplish a task by using either Windows Runtime type or a type provided by .NET for UWP apps. It is a good idea to try to not mix .NET types and Windows Runtime types. Try to stay in one or the other. For example, you can parse a stream of xml by using either the Windows.Data.Xml.Dom.XmlDocument type (a Windows Runtime type) or the

System.Xml.XmlReader type (a .NET type). Use the API that is from the same technology as the stream. For example, if you read xml from a MemoryStream, use the **System.Xml.XmlReader** type, because both types are .NET types. If you read from a file, use the **Windows.Data.Xml.Dom.XmlDocument** type because the file APIs and **XmlDocument** are both implemented in native Windows Runtime components.

Copy Window Runtime objects to .NET types

When a Windows Runtime component returns a Windows Runtime object, it might be beneficial to copy the returned object into a .NET object. Two places where this is especially important is when you're working with collections and streams.

If you call a Windows Runtime API that returns a collection and then you save and access that collection many times, it might be beneficial to copy the collection into a .NET collection and use the .NET version from then on.

Cache the results of calls to Windows Runtime components for later use

You might be able to get better performance by saving values into local variables instead of accessing a Windows Runtime type multiple times. This can be especially beneficial if you use a value inside of a loop. Measure your app to see if using local variables improves your app's performance. Using cached values can increase your app's speed because it will spend less time on interoperability.

Combine calls to Windows Runtime components

Try to complete tasks with the fewest number of calls to UWP objects as possible. For example, it is usually better to read a large amount of data from a stream than to read small amounts at a time.

Use APIs that bundle work in as few calls as possible instead of APIs that do less work and require more calls. For example, prefer to create an object by calling constructors that initialize multiple properties once instead of calling the default constructor and assigning properties one at a time.

Building a Windows Runtime components

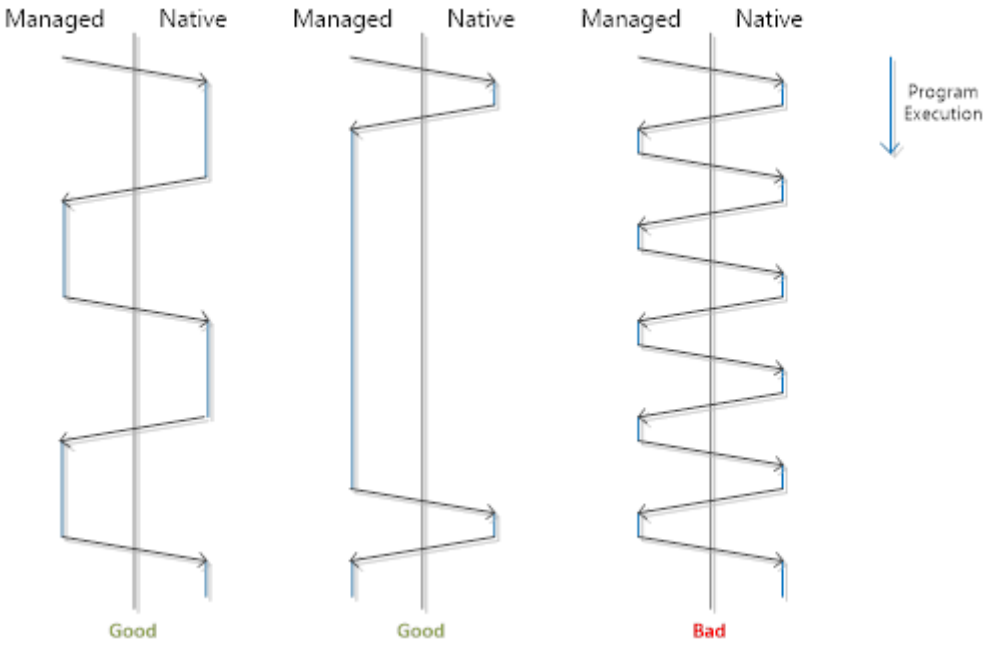
If you write a Windows Runtime Component that can be used by apps written in C++ or JavaScript, make sure that your component is designed for good performance. All the suggestions for getting good performance in apps apply to getting good performance

in components. Measure your component to find out which APIs have high traffic patterns and for those areas, consider providing APIs that enable your users to do work with few calls.

Keep your app fast when you use interop in managed code

The Windows Runtime makes it easy to interoperate between native and managed code, but if you're not careful it can incur performance costs. Here we show you how to get good performance when you use interop in your managed UWP apps.

The Widows Runtime allows developers to write apps using XAML with their language of choice thanks to the projections of the Windows Runtime APIs available in each language. When writing an app in C# or Visual Basic, this convenience comes at an interop cost because the Windows Runtime APIs are usually implemented in native code, and any Windows Runtime invocation from C# or Visual Basic requires that the CLR transition from a managed to a native stack frame and marshal function parameters to representations accessible by native code. This overhead is negligible for most apps. But when you make many calls (hundreds of thousands, to millions) to Windows Runtime APIs in the critical path of an app, this cost can become noticeable. In general you want to ensure that the time spent in transition between languages is small relative to the execution of the rest of your code. This is illustrated by the following diagram.



The types listed at **.NET for Windows apps** don't incur this interop cost when used from C# or Visual Basic. As a rule of thumb, you can assume that types in namespaces which begin with “Windows.” are part of the Windows Runtime API set provided by Windows, and types in namespaces which begin with “System.” are .NET types. Keep in

mind that even simple usage of Windows Runtime types incurs an interop cost for allocation or property access.

You should measure your app and determine if interop is taking up a large portion of your apps execution time before optimizing your interop costs. When analyzing your app's performance with Visual Studio, you can easily get an upper bound on your interop costs by using the **Functions** view and looking at inclusive time spent in methods which call into the Windows Runtime.

If your app is slow because of interop overhead, you can improve its performance by reducing calls to Windows Runtime APIs on hot code paths. For example, a game engine that is doing tons of physics calculations by constantly querying the position and dimensions of **UIElements** can save a lot of time by storing the necessary info from **UIElements** to local variables, doing calculations on these cached values, and assigning the end result back to the **UIElements** after the calculations are done. Another example: if a collection is heavily accessed by C# or Visual Basic code, then it is more efficient to use a collection from the **System.Collections** namespace, rather than a collection from the **Windows.Foundation.Collections** namespace. You may also consider combining calls to Windows Runtime components; one example where this is possible is by using the **Windows.Storage.BulkAccess** APIs.

Building a UWP component

If you write a Windows Runtime component for use in apps written in C++ or JavaScript, make sure that your component is designed for good performance. Your API surface defines your interop boundary and defines the degree to which your users will have to think about the guidance in this topic. If you are distributing your components to other parties then this becomes especially important.

All of the suggestions for getting good performance in apps apply to getting good performance in components. Measure your component to find out which APIs have high traffic patterns, and for those areas, consider providing APIs that enable your users to do work with few calls. Significant effort was put into designing the Windows Runtime to allow apps to use it without requiring frequent crossing of the interop boundary.

Feedback

Was this page helpful? Yes No

Tools for profiling and performance

Article • 10/20/2022

Microsoft provides several tools to help you improve the performance of your Universal Windows Platform (UWP) app. Follow these links to learn how to use these tools.

XAML UI Responsiveness tool in Visual Studio. One of the best tools to use for measuring performance impact within your app is the XAML UI Responsiveness Tool. This has been updated to support even more scenarios.

See also:

- Analyze the performance of UWP apps using Visual Studio diagnostic tools: By showing you where the code of your app spends its time as your program executes, the Visual Studio profilers can help you find the performance bottlenecks in your apps, functions, and algorithms.
- XAML Performance: Techniques for Maximizing Universal Windows App Experiences Built with XAML: In this //build session, you will learn about the new platform features, new tooling features and new techniques to dramatically increase the performance of your XAML-based Universal Windows app.
- New XAML Tools in Visual Studio: In this //build session, you will learn about some of the new capabilities in Visual Studio 2015, including the re-designed Blend experience, UI debugging tools and the XAML editor enhancements. These tools are also available in Visual Studio 2019.

Version adaptive apps: Use new APIs while maintaining compatibility with previous versions

Article • 03/29/2023

Each release of the Windows 10 SDK adds exciting new functionality that you'll want to take advantage of. However, not all your customers will update their devices to the latest version of Windows 10 at the same time, and you want to make sure your app works on the broadest possible range of devices. Here, we show you how to design your app so that it runs on earlier versions of Windows 10, but also takes advantage of new features whenever your app runs on a device with the latest update installed.

There are 3 steps to take to make sure your app supports the broadest range of Windows 10 devices.

- First, configure your Visual Studio project to target the latest APIs. This affects what happens when you compile your app.
- Second, perform runtime checks to ensure that you only call APIs that are present on the device your app is running on.
- Third, test your app on the Minimum Version and the Target Version of Windows 10.

Configure your Visual Studio project

The first step in supporting multiple Windows 10 versions is to specify the *Target* and *Minimum* supported OS/SDK versions in your Visual Studio project.

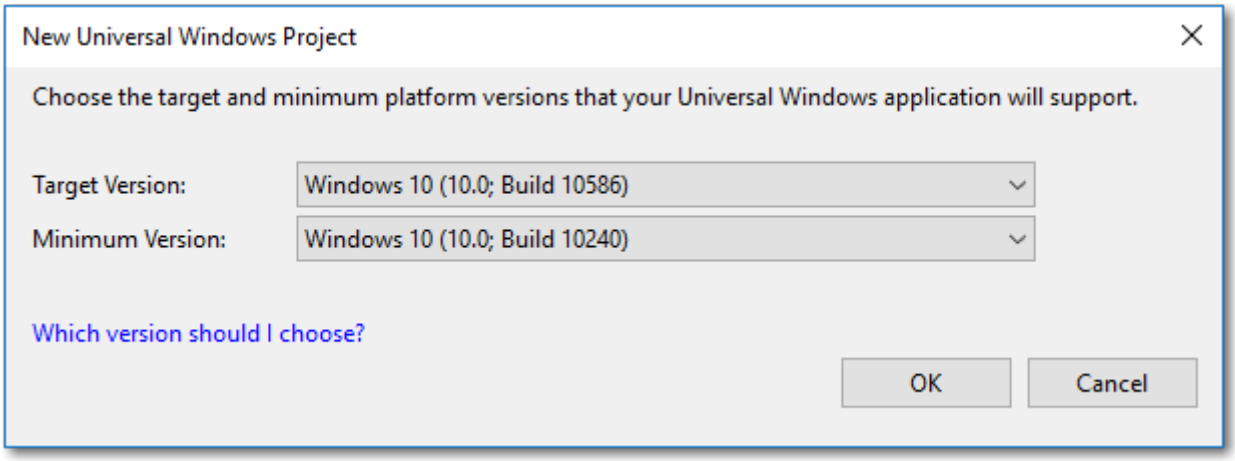
- *Target*: The SDK version that Visual Studio compiles your app code and run all tools against. All APIs and resources in this SDK version are available in your app code at compile time.
- *Minimum*: The SDK version that supports the earliest OS version that your app can run on (and will be deployed to by the store) and the version that Visual Studio compiles your app markup code against.

During runtime your app will run against the OS version that it is deployed to, so your app will throw exceptions if you use resources or call APIs that are not available in that version. We show you how to use runtime checks to call the correct APIs later in this article.

The Target and Minimum settings specify the ends of a range of OS/SDK versions. However, if you test your app on the minimum version, you can be sure it will run on any versions between the Minimum and Target.

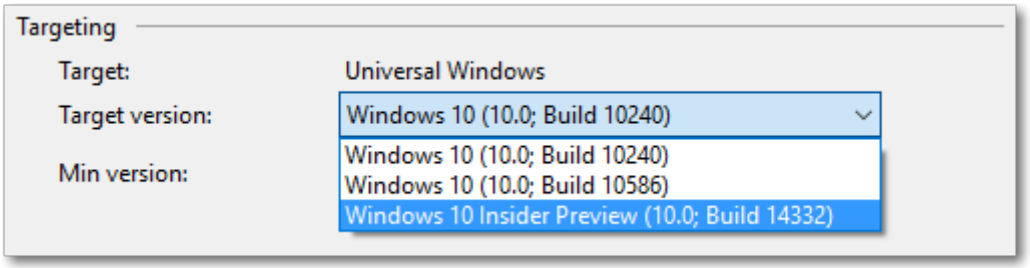
> **Tip**
>
> Visual Studio does not warn you about API compatibility. It is your responsibility to test and ensure that your app performs as expected on all OS versions between and including the Minimum and Target.

When you create a new project in Visual Studio 2015, Update 2 or later, you are prompted to set the Target and Minimum versions that your app supports. By default, the Target Version is the highest installed SDK version, and the Minimum Version is the lowest installed SDK version. You can choose Target and Minimum only from SDK versions that are installed on your machine.



We typically recommend that you leave the defaults. However, if you have a Preview version of the SDK installed and you are writing production code, you should change the Target Version from the Preview SDK to the latest official SDK version.

To change the Minimum and Target version for a project that has already been created in Visual Studio, go to Project -> Properties -> Application tab -> Targeting.



For reference, the following table shows the build numbers for each SDK.

Friendly name	Version	OS/SDK build
RTM	1507	10240
November Update	1511	10586
Anniversary Update	1607	14393
Creators Update	1703	15063
Fall Creators Update	1709	16299
April 2018 Update	1803	17134
October 2018 Update	1809	17763
May 2019 Update	1903	18362

You can download any released version of the SDK from the Windows SDK and emulator archive. You can download the latest Windows Insider Preview SDK from the developer section of the Windows Insider site.

For more info about Windows 10 updates, see Windows 10 release information. For important information about the Windows 10 support lifecycle, see the Windows lifecycle fact sheet.

Perform API checks

The key to version adaptive apps is the combination of API contracts and the ApiInformation class. This class lets you detect whether a specified API contract, type, or member is present so that you can safely make API calls across a variety of devices and OS versions.

API contracts

The set of APIs within a device family is broken down into subdivisions known as API contracts. You can use the **ApiInformation.IsApiContractPresent** method to test for the presence of an API contract. This is useful if you want to test for the presence of many APIs that all exist in the same version of an API contract.

C#

```
    bool isScannerDeviceContract_1_Present =
        Windows.Foundation.Metadata.ApiInformation.IsApiContractPresent
            ("Windows.Devices.Scanners.ScannerDeviceContract", 1);
```

What's an API contract? Essentially, an API contract represents a feature – a set of related APIs that together deliver some particular functionality. A hypothetical API contract could represent a set of APIs containing two classes, five interfaces, one structure, two enums, and so on.

Logically related types are grouped into an API contract, and starting with Windows 10, every Windows Runtime API is a member of some API contract. With API Contracts, you are checking for the availability of a specific feature or API on the device, effectively checking a device's capabilities rather than checking for a specific device or OS. A platform that implements any API in an API contract is required to implement every API in that API contract. This means you can test whether the running OS supports a particular API contract and, if it does, call any of the APIs in that API contract without checking each one individually.

The largest and most commonly used API contract is the **Windows.Foundation.UniversalApiContract**. It contains the majority of the APIs in the Universal Windows Platform. The Device family extension SDKs and API contracts documentation describes the variety of API contracts available. You'll see that most of them represent a set of functionally related APIs.

> **Note**
>
> If you have a preview Windows Software Development Kit (SDK) installed that isn't documented yet, you can also find information about API contract support in the 'Platform.xml' file located in the SDK installation folder at '(Program Files (x86))\Windows Kits\10\Platforms<platform><SDK version>\Platform.xml'.

Version adaptive code and conditional XAML

In all versions of Windows 10, you can use the ApiInformation class in a condition in your code to test for the presence of the API you want to call. In your adaptive code, you can use various methods of the class, such as IsTypePresent, IsEventPresent, IsMethodPresent, and IsPropertyPresent, to test for APIs at the granularity you need.

For more info and examples, see **Version adaptive code**.

If your apps Minimum Version is build 15063 (Creators Update) or later, you can use *conditional XAML* to set properties and instantiate objects in markup without needing to use code behind. Conditional XAML provides a way to use the ApiInformation.IsApiContractPresent method in markup.

For more info and examples, see **Conditional XAML**.

Delay-loaded APIs

The QueryOptionalDelayLoadedAPI function enables Store apps (which are not allowed by policy to call LoadLibrary) to detect whether their delay-loaded function actually was found.

C++

```
#include <windows.h>
#include <commdlg.h>
#include <libloaderapi2.h>
#include <stdio.h>

EXTERN_C IMAGE_DOS_HEADER __ImageBase;
#define HMODULE_THISCOMPONENT reinterpret_cast<HMODULE>(&__ImageBase)

int __cdecl main(int argc, char** argv)
{
    if (QueryOptionalDelayLoadedAPI(HMODULE_THISCOMPONENT,
          "comdlg32.dll", "GetOpenFileNameW", 0))
    {
        printf("GetOpenFileNameW can be called!\n");
    }
    return 0;
}
```

Test your version adaptive app

When you use version adaptive code or conditional XAML to write a version adaptive app, you need to test it on a device running the Minimum Version and on a device running the Target Version of Windows 10.

You can't test all conditional code paths on a single device. To ensure that all code paths are tested, you need to deploy and test your app on a remote device (or virtual machine) running the minimum supported OS version. For more info about remote debugging, see Deploying and debugging UWP apps.

Related articles

- What's a UWP app

Version adaptive code

Article • 03/29/2023

You can think about writing adaptive code similarly to how you think about creating an adaptive UI. You might design your base UI to run on the smallest screen, and then move or add elements when you detect that your app is running on a larger screen. With adaptive code, you write your base code to run on the lowest OS version, and you can add hand-selected features when you detect that your app is running on a higher version where the new feature is available.

For important background info about ApiInformation, API contracts, and configuring Visual Studio, see Version adaptive apps.

Runtime API checks

You use the Windows.Foundation.Metadata.ApiInformation class in a condition in your code to test for the presence of the API you want to call. This condition is evaluated wherever your app runs, but it evaluates to **true** only on devices where the API is present and therefore available to call. This lets you write version adaptive code in order to create apps that use APIs that are available only on certain OS versions.

Here, we look at specific examples for targeting new features in the Windows Insider Preview. For a general overview of using **ApiInformation**, see Programming with extension SDKs and Dynamically detecting features with API contracts.

> **Tip**
>
> Numerous runtime API checks can affect the performance of your app. We show the checks inline in these examples. In production code, you should perform the check once and cache the result, then used the cached result throughout your app.

Unsupported scenarios

In most cases, you can keep your app's Minimum Version set to SDK version 10240 and use runtime checks to enable any new APIs when your app runs on later a version. However, there are some cases where you must increase your app's Minimum Version in order to use new features.

You must increase your app's Minimum Version if you use:

- a new API that requires a capability that isn't available in an earlier version. You must increase the minimum supported version to one that includes that capability. For more info, see App capability declarations.
- any new resource keys added to generic.xaml and not available in a previous version. The version of generic.xaml used at runtime is determined by the OS version the device is running on. You can't use runtime API checks to determine the presence of XAML resources. So, you must only use resource keys that are available in the minimum version that your app supports or a **XAMLParseException** will cause your app to crash at runtime.

Adaptive code options

There are two ways to create adaptive code. In most cases, you write your app markup to run on the Minimum version, then use your app code to tap into newer OS features when present. However, if you need to update a property in a visual state, and there is only a property or enumeration value change between OS versions, you can create an extensible state trigger that's activated based on the presence of an API.

Here, we compare these options.

App code

When to use:

- Recommended for all adaptive code scenarios except for specific cases defined below for extensible triggers.

Benefits:

- Avoids developer overhead/complexity of tying API differences into markup.

Drawbacks:

- No Designer support.

State Triggers

When to use:

- Use when there is only a property or enum change between OS versions that doesn't require logic changes, and is connected to a visual state.

Benefits:

- Lets you create specific visual states that are triggered based on the presence of an API.
- Some designer support available.

Drawbacks:

- Use of custom triggers is restricted to visual states, which doesn't lend itself to complicated adaptive layouts.
- Must use Setters to specify value changes, so only simple changes are possible.
- Custom state triggers are fairly verbose to set up and use.

Adaptive code examples

In this section, we show several examples of adaptive code that use APIs that are new in Windows 10, version 1607 (Windows Insider Preview).

Example 1: New enum value

Windows 10, version 1607 adds a new value to the InputScopeNameValue enumeration: **ChatWithoutEmoji**. This new input scope has the same input behavior as the **Chat** input scope (spellchecking, auto-complete, auto-capitalization), but it maps to a touch keyboard without an emoji button. This is useful if you create your own emoji picker and want to disable the built-in emoji button in the touch keyboard.

This example shows how to check if the **ChatWithoutEmoji** enum value is present and sets the InputScope property of a **TextBox** if it is. If it's not present on the system the app is run on, the **InputScope** is set to **Chat** instead. The code shown could be placed in a Page constructor or Page.Loaded event handler.

> **Tip**
>
> When you check an API, use static strings instead of relying on .NET language features, otherwise your app might try to access a type that isn't defined and crash at runtime.

C#

```
C#

// Create a TextBox control for sending messages
// and initialize an InputScope object.
TextBox messageBox = new TextBox();
messageBox.AcceptsReturn = true;
```

```
messageBox.TextWrapping = TextWrapping.Wrap;
InputScope scope = new InputScope();
InputScopeName scopeName = new InputScopeName();

// Check that the ChatWithEmoji value is present.
// (It's present starting with Windows 10, version 1607,
//  the Target version for the app. This check returns false on earlier
versions.)
if
(ApiInformation.IsEnumNamedValuePresent("Windows.UI.Xaml.Input.InputScopeNam
eValue", "ChatWithoutEmoji"))
{
    // Set new ChatWithoutEmoji InputScope if present.
    scopeName.NameValue = InputScopeNameValue.ChatWithoutEmoji;
}
else
{
    // Fall back to Chat InputScope.
    scopeName.NameValue = InputScopeNameValue.Chat;
}

// Set InputScope on messaging TextBox.
scope.Names.Add(scopeName);
messageBox.InputScope = scope;

// For this example, set the TextBox text to show the selected InputScope.
messageBox.Text = messageBox.InputScope.Names[0].NameValue.ToString();

// Add the TextBox to the XAML visual tree (rootGrid is defined in XAML).
rootGrid.Children.Add(messageBox);
```

In the previous example, the TextBox is created and all properties are set in code. However, if you have existing XAML, and just need to change the InputScope property on systems where the new value is supported, you can do that without changing your XAML, as shown here. You set the default value to **Chat** in XAML, but you override it in code if the **ChatWithoutEmoji** value is present.

XAML

XAML

```
<TextBox x:Name="messageBox"
         AcceptsReturn="True" TextWrapping="Wrap"
         InputScope="Chat"
         Loaded="messageBox_Loaded"/>
```

C#

C#

```
private void messageBox_Loaded(object sender, RoutedEventArgs e)
{
    if
(ApiInformation.IsEnumNamedValuePresent("Windows.UI.Xaml.Input.InputScopeNam
eValue", "ChatWithoutEmoji"))
    {
        // Check that the ChatWithEmoji value is present.
        // (It's present starting with Windows 10, version 1607,
        //  the Target version for the app. This code is skipped on earlier
versions.)
        InputScope scope = new InputScope();
        InputScopeName scopeName = new InputScopeName();
        scopeName.NameValue = InputScopeNameValue.ChatWithoutEmoji;
        // Set InputScope on messaging TextBox.
        scope.Names.Add(scopeName);
        messageBox.InputScope = scope;
    }

    // For this example, set the TextBox text to show the selected
InputScope.
    // This is outside of the API check, so it will happen on all OS
versions.
    messageBox.Text = messageBox.InputScope.Names[0].NameValue.ToString();
}
```

Now that we have a concrete example, let's see how the Target and Minimum version settings apply to it.

In these examples, you can use the Chat enum value in XAML, or in code without a check, because it's present in the minimum supported OS version.

If you use the ChatWithoutEmoji value in XAML, or in code without a check, it will compile without error because it's present in the Target OS version. It will also run without error on a system with the Target OS version. However, when the app runs on a system with an OS using the Minimum version, it will crash at runtime because the ChatWithoutEmoji enum value is not present. Therefore, you must use this value only in code, and wrap it in a runtime API check so it's called only if it's supported on the current system.

Example 2: New control

A new version of Windows typically brings new controls to the UWP API surface that bring new functionality to the platform. To leverage the presence of a new control, use the ApiInformation.IsTypePresent method.

Windows 10, version 1607 introduces a new media control called **MediaPlayerElement**. This control builds on the MediaPlayer class, so it brings features like the ability to easily

tie into background audio, and it makes use of architectural improvements in the media stack.

However, if the app runs on a device that's running a version of Windows 10 older than version 1607, you must use the **MediaElement** control instead of the new **MediaPlayerElement** control. You can use the **ApiInformation.IsTypePresent** method to check for the presence of the MediaPlayerElement control at runtime, and load whichever control is suitable for the system where the app is running.

This example shows how to create an app that uses either the new MediaPlayerElement or the old MediaElement depending on whether MediaPlayerElement type is present. In this code, you use the UserControl class to componentize the controls and their related UI and code so that you can switch them in and out based on the OS version. As an alternative, you can use a custom control, which provides more functionality and custom behavior than what's needed for this simple example.

MediaPlayerUserControl

The `MediaPlayerUserControl` encapsulates a **MediaPlayerElement** and several buttons that are used to skip through the media frame by frame. The UserControl lets you treat these controls as a single entity and makes it easier to switch with a MediaElement on older systems. This user control should be used only on systems where MediaPlayerElement is present, so you don't use ApiInformation checks in the code inside this user control.

> **Note**
>
> To keep this example simple and focused, the frame step buttons are placed outside of the media player. For a better user experience, you should customize the MediaTransportControls to include your custom buttons. See Custom transport controls for more info.

XAML

XAML

```
<UserControl
    x:Class="MediaApp.MediaPlayerUserControl"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:MediaApp"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    d:DesignHeight="300"
```

```
    d:DesignWidth="400">

    <Grid x:Name="MPE_grid>
        <Grid.RowDefinitions>
            <RowDefinition/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <StackPanel Orientation="Horizontal"
                    HorizontalAlignment="Center" Grid.Row="1">
            <RepeatButton Click="StepBack_Click" Content="Step Back"/>
            <RepeatButton Click="StepForward_Click" Content="Step Forward"/>
        </StackPanel>
    </Grid>
</UserControl>
```

C#

C#

```
using System;
using Windows.Media.Core;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;

namespace MediaApp
{
    public sealed partial class MediaPlayerUserControl : UserControl
    {
        public MediaPlayerUserControl()
        {
            this.InitializeComponent();

            // The markup code compiler runs against the Minimum OS version
so MediaPlayerElement must be created in app code
            MPE = new MediaPlayerElement();
            Uri videoSource = new Uri("ms-appx:///Assets/UWPDesign.mp4");
            MPE.Source = MediaSource.CreateFromUri(videoSource);
            MPE.AreTransportControlsEnabled = true;
            MPE.MediaPlayer.AutoPlay = true;

            // Add MediaPlayerElement to the Grid
            MPE_grid.Children.Add(MPE);

        }

        private void StepForward_Click(object sender, RoutedEventArgs e)
        {
            // Step forward one frame, only available using
MediaPlayerElement.
            MPE.MediaPlayer.StepForwardOneFrame();
        }

        private void StepBack_Click(object sender, RoutedEventArgs e)
```

```
        {
            // Step forward one frame, only available using
MediaPlayerElement.
            MPE.MediaPlayer.StepForwardOneFrame();
        }
    }
}
```

MediaElementUserControl

The `MediaElementUserControl` encapsulates a **MediaElement** control.

XAML

XAML

```
<UserControl
    x:Class="MediaApp.MediaElementUserControl"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:MediaApp"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    d:DesignHeight="300"
    d:DesignWidth="400">

    <Grid>
        <MediaElement AreTransportControlsEnabled="True"
                      Source="Assets/UWPDesign.mp4"/>
    </Grid>
</UserControl>
```

> **Note**
>
> The code page for `MediaElementUserControl` contains only generated code, so it's not shown.

Initialize a control based on IsTypePresent

At runtime, you call **ApiInformation.IsTypePresent** to check for MediaPlayerElement. If it's present, you load `MediaPlayerUserControl`, if it's not, you load `MediaElementUserControl`.

C#

C#

```
public MainPage()
{
    this.InitializeComponent();

    UserControl mediaControl;

    // Check for presence of type MediaPlayerElement.
    if
(ApiInformation.IsTypePresent("Windows.UI.Xaml.Controls.MediaPlayerElement")
)
    {
        mediaControl = new MediaPlayerUserControl();
    }
    else
    {
        mediaControl = new MediaElementUserControl();
    }

    // Add mediaControl to XAML visual tree (rootGrid is defined in XAML).
    rootGrid.Children.Add(mediaControl);
}
```

> **ⓘ Important**
>
> Remember that this check only sets the `mediaControl` object to either `MediaPlayerUserControl` or `MediaElementUserControl`. You need to perform these conditional checks anywhere else in your code that you need to determine whether to use MediaPlayerElement or MediaElement APIs. You should perform the check once and cache the result, then used the cached result throughout your app.

State trigger examples

Extensible state triggers let you use markup and code together to trigger visual state changes based on a condition that you check in code; in this case, the presence of a specific API. We don't recommend state triggers for common adaptive code scenarios because of the overhead involved, and the restriction to only visual states.

You should use state triggers for adaptive code only when you have small UI changes between different OS versions that won't impact the remaining UI, such as a property or enum value change on a control.

Example 1: New property

The first step in setting up an extensible state trigger is subclassing the StateTriggerBase class to create a custom trigger that will be active based on the presence of an API. This example shows a trigger that activates if the property presence matches the `_isPresent` variable set in XAML.

C#

```
C#

class IsPropertyPresentTrigger : StateTriggerBase
{
    public string TypeName { get; set; }
    public string PropertyName { get; set; }

    private Boolean _isPresent;
    private bool? _isPropertyPresent = null;

    public Boolean IsPresent
    {
        get { return _isPresent; }
        set
        {
            _isPresent = value;
            if (_isPropertyPresent == null)
            {
                // Call into ApiInformation method to determine if property
is present.
                _isPropertyPresent =
                ApiInformation.IsPropertyPresent(TypeName, PropertyName);
            }

            // If the property presence matches _isPresent then the trigger
will be activated;
            SetActive(_isPresent == _isPropertyPresent);
        }
    }
}
```

The next step is setting up the visual state trigger in XAML so that two different visual states result based on the presence of the API.

Windows 10, version 1607 introduces a new property on the FrameworkElement class called AllowFocusOnInteraction that determines whether a control takes focus when a user interacts with it. This is useful if you want to keep focus on a text box for data entry (and keep the touch keyboard showing) while the user clicks a button.

The trigger in this example checks if the property is present. If the property is present it sets the **AllowFocusOnInteraction** property on a Button to **false**; if the property isn't

present, the Button retains its original state. The TextBox is included to make it easier to see the effect of this property when you run the code.

XAML

XAML

```
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
    <StackPanel>
        <TextBox Width="300" Height="36"/>
        <!-- Button to set the new property on. -->
        <Button x:Name="testButton" Content="Test" Margin="12"/>
    </StackPanel>

    <VisualStateManager.VisualStateGroups>
        <VisualStateGroup x:Name="propertyPresentStateGroup">
            <VisualState>
                <VisualState.StateTriggers>
                    <!--Trigger will activate if the AllowFocusOnInteraction
property is present-->
                    <local:IsPropertyPresentTrigger
                        TypeName="Windows.UI.Xaml.FrameworkElement"
                        PropertyName="AllowFocusOnInteraction"
IsPresent="True"/>
                </VisualState.StateTriggers>
                <VisualState.Setters>
                    <Setter Target="testButton.AllowFocusOnInteraction"
                            Value="False"/>
                </VisualState.Setters>
            </VisualState>
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
</Grid>
```

Example 2: New enum value

This example shows how to set different enumeration values based on whether a value is present. It uses a custom state trigger to achieve the same result as the previous chat example. In this example, you use the new ChatWithoutEmoji input scope if the device is running Windows 10, version 1607, otherwise the **Chat** input scope is used. The visual states that use this trigger are set up in an *if-else* style where the input scope is chosen based on the presence of the new enum value.

C#

C#

```
class IsEnumPresentTrigger : StateTriggerBase
{
```

```
    public string EnumTypeName { get; set; }
    public string EnumValueName { get; set; }

    private Boolean _isPresent;
    private bool? _isEnumValuePresent = null;

    public Boolean IsPresent
    {
        get { return _isPresent; }
        set
        {
            _isPresent = value;

            if (_isEnumValuePresent == null)
            {
                // Call into ApiInformation method to determine if value is
present.
                _isEnumValuePresent =
                ApiInformation.IsEnumNamedValuePresent(EnumTypeName,
EnumValueName);
            }

            // If the property presence matches _isPresent then the trigger
will be activated;
            SetActive(_isPresent == _isEnumValuePresent);
        }
    }
}
```

XAML

XAML

```
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">

    <TextBox x:Name="messageBox"
     AcceptsReturn="True" TextWrapping="Wrap"/>


    <VisualStateManager.VisualStateGroups>
        <VisualStateGroup x:Name="EnumPresentStates">
            <!--if-->
            <VisualState x:Name="isPresent">
                <VisualState.StateTriggers>
                    <local:IsEnumPresentTrigger

EnumTypeName="Windows.UI.Xaml.Input.InputScopeNameValue"
                                EnumValueName="ChatWithoutEmoji" IsPresent="True"/>
                </VisualState.StateTriggers>
                <VisualState.Setters>
                    <Setter Target="messageBox.InputScope"
Value="ChatWithoutEmoji"/>
                </VisualState.Setters>
```

```
            </VisualState>
            <!--else-->
            <VisualState x:Name="isNotPresent">
                <VisualState.StateTriggers>
                    <local:IsEnumPresentTrigger

EnumTypeName="Windows.UI.Xaml.Input.InputScopeNameValue"
                        EnumValueName="ChatWithoutEmoji" IsPresent="False"/>
                </VisualState.StateTriggers>
                <VisualState.Setters>
                    <Setter Target="messageBox.InputScope" Value="Chat"/>
                </VisualState.Setters>
            </VisualState>
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
</Grid>
```

Related articles

- Programming with extension SDKs
- Dynamically detecting features with API contracts

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Conditional XAML

Article • 03/29/2023

Conditional XAML provides a way to use the ApiInformation.IsApiContractPresent method in XAML markup. This lets you set properties and instantiate objects in markup based on the presence of an API without needing to use code behind. It selectively parses elements or attributes to determine whether they will be available at runtime. Conditional statements are evaluated at runtime, and elements qualified with a conditional XAML tag are parsed if they evaluate to **true**; otherwise, they are ignored.

Conditional XAML is available starting with the Creators Update (version 1703, build 15063). To use conditional XAML, the Minimum Version of your Visual Studio project must be set to build 15063 (Creators Update) or later, and the Target Version be set to a later version than the Minimum. See Version adaptive apps for more info about configuring your Visual Studio project.

> **Note**
>
> To create a version adaptive app with a Minimum Version less than build 15063, you must use **version adaptive code**, not XAML.

For important background info about ApiInformation and API contracts, see Version adaptive apps.

Conditional namespaces

To use a conditional method in XAML, you must first declare a conditional XAML namespace at the top of your page. Here's a pseudo-code example of a conditional namespace:

XAML

```
xmlns:myNamespace="schema?conditionalMethod(parameter)"
```

A conditional namespace can be broken down into two parts separated by the '?' delimiter.

- The content preceding the delimiter indicates the namespace or schema that contains the API being referenced.

- The content after the '?' delimiter represents the conditional method that determines whether the conditional namespace evaluates to **true** or **false**.

In most cases, the schema will be the default XAML namespace:

```
XAML
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

Conditional XAML supports the following conditional methods:

Method	Inverse
IsApiContractPresent(ContractName, VersionNumber)	IsApiContractNotPresent(ContractName, VersionNumber)
IsTypePresent(ControlType)	IsTypeNotPresent(ControlType)
IsPropertyPresent(ControlType, PropertyName)	IsPropertyNotPresent(ControlType, PropertyName)

We discuss these methods further in later sections of this article.

> ⓘ **Note**
>
> We recommend you use IsApiContractPresent and IsApiContractNotPresent. Other conditionals are not fully supported in the Visual Studio design experience.

Create a namespace and set a property

In this example, you display, "Hello, Conditional XAML", as the content of a text block if the app runs on the Fall Creators Update or later, and default to no content if it's on a previous version.

First, define a custom namespace with the prefix 'contract5Present' and use the default XAML namespace (`https://schemas.microsoft.com/winfx/2006/xaml/presentation`) as the schema containing the TextBlock.Text property. To make this a conditional namespace, add the '?' delimiter after the schema.

You then define a conditional that returns **true** on devices that are running the Fall Creators Update or later. You use the ApiInformation method **IsApiContractPresent** to check for the 5th version of the UniversalApiContract. Version 5 of the UniversalApiContract was released with the Fall Creators Update (SDK 16299).

XAML

```
xmlns:contract5Present="http://schemas.microsoft.com/winfx/2006/xaml/present
ation?IsApiContractPresent(Windows.Foundation.UniversalApiContract,5)"
```

After the namespace is defined, you prepend the namespace prefix to the Text property of your TextBox to qualify it as a property that should be set conditionally at runtime.

XAML

```
<TextBlock contract5Present:Text="Hello, Conditional XAML"/>
```

Here's the complete XAML.

XAML

```
<Page
    x:Class="ConditionalTest.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:contract5Present="http://schemas.microsoft.com/winfx/2006/xaml/present
ation?IsApiContractPresent(Windows.Foundation.UniversalApiContract,5)">

    <Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
        <TextBlock contract5Present:Text="Hello, Conditional XAML"/>
    </Grid>
</Page>
```

When you run this example on the Fall Creators Update, the text, "Hello, Conditional XAML" is shown; when you run it on the Creators Update, no text is shown.

Conditional XAML lets you perform the API checks you can do in code in your markup instead. Here's the equivalent code for this check.

C#

```
if
(ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 5))
{
    textBlock.Text = "Hello, Conditional XAML";
}
```

Notice that even though the IsApiContractPresent method takes a string for the *contractName* parameter, you don't put it in quotes (" ") in the XAML namespace

declaration.

Use if/else conditions

In the previous example, the Text property is set only when the app runs on the Fall Creators Update. But what if you want to show different text when it runs on the Creators Update? You could try to set the Text property without a conditional qualifier, like this.

XAML

```
<!-- DO NOT USE -->
<TextBlock Text="Hello, World" contract5Present:Text="Hello, Conditional
XAML"/>
```

This will work when it runs on the Creators Update, but when it runs on the Fall Creators Update, you get an error saying that the Text property is set more than once.

To set different text when the app runs on different versions of Windows 10, you need another condition. Conditional XAML provides an inverse of each supported ApiInformation method to let you create if/else conditional scenarios like this.

The IsApiContractPresent method returns **true** if the current device contains the specified contract and version number. For example, assume your app is running on the Creators Update, which has the 4th version of the universal API Contract.

Various calls to IsApiContractPresent would have these results:

- IsApiContractPresent(Windows.Foundation.UniversalApiContract, 5) = **false**
- IsApiContractPresent(Windows.Foundation.UniversalApiContract, 4) = true
- IsApiContractPresent(Windows.Foundation.UniversalApiContract, 3) = true
- IsApiContractPresent(Windows.Foundation.UniversalApiContract, 2) = true
- IsApiContractPresent(Windows.Foundation.UniversalApiContract, 1) = true.

IsApiContractNotPresent returns the inverse of IsApiContractPresent. Calls to IsApiContractNotPresent would have these results:

- IsApiContractNotPresent(Windows.Foundation.UniversalApiContract, 5) = **true**
- IsApiContractNotPresent(Windows.Foundation.UniversalApiContract, 4) = false
- IsApiContractNotPresent(Windows.Foundation.UniversalApiContract, 3) = false
- IsApiContractNotPresent(Windows.Foundation.UniversalApiContract, 2) = false
- IsApiContractNotPresent(Windows.Foundation.UniversalApiContract, 1) = false

To use the inverse condition, you create a second conditional XAML namespace that uses the **IsApiContractNotPresent** conditional. Here, it has the prefix 'contract5NotPresent'.

XAML

```
xmlns:contract5NotPresent="http://schemas.microsoft.com/winfx/2006/xaml/pres
entation?IsApiContractNotPresent(Windows.Foundation.UniversalApiContract,5)"

xmlns:contract5Present="http://schemas.microsoft.com/winfx/2006/xaml/present
ation?IsApiContractPresent(Windows.Foundation.UniversalApiContract,5)"
```

With both namespaces defined, you can set the Text property twice as long as you prefix them with qualifiers that ensure only one property setting is used at runtime, like this:

XAML

```
<TextBlock contract5NotPresent:Text="Hello, World"
           contract5Present:Text="Hello, Fall Creators Update"/>
```

Here's another example that sets the background of a button. The Acrylic material feature is available starting with the Fall Creators Update, so you'll use Acrylic for the background when the app runs on the Fall Creators Update. It's not available on earlier versions, so in those cases, you set the background to red.

XAML

```
<Button Content="Button"
        contract5NotPresent:Background="Red"
        contract5Present:Background="{ThemeResource
SystemControlAcrylicElementBrush}"/>
```

Create controls and bind properties

So far, you've seen how to set properties using conditional XAML, but you can also conditionally instantiate controls based on the API contract available at runtime.

Here, a ColorPicker is instantiated when the app runs on the Fall Creators Update where the control is available. The ColorPicker isn't available prior to the Fall Creators Update, so when the app runs on earlier versions, you use a ComboBox to provide simplified color choices to the user.

XAML

```
<contract5Present:ColorPicker x:Name="colorPicker"
                              Grid.Column="1"
                              VerticalAlignment="Center"/>

<contract5NotPresent:ComboBox x:Name="colorComboBox"
                              PlaceholderText="Pick a color"
                              Grid.Column="1"
                              VerticalAlignment="Center">
```

You can use conditional qualifiers with different forms of XAML property syntax. Here, the rectangle's Fill property is set using property element syntax for the Fall Creators Update, and using attribute syntax for previous versions.

XAML

```
<Rectangle x:Name="colorRectangle" Width="200" Height="200"
           contract5NotPresent:Fill="{x:Bind ((SolidColorBrush)
((FrameworkElement)colorComboBox.SelectedItem).Tag), Mode=OneWay}">
    <contract5Present:Rectangle.Fill>
        <SolidColorBrush contract5Present:Color="{x:Bind colorPicker.Color,
Mode=OneWay}"/>
    </contract5Present:Rectangle.Fill>
</Rectangle>
```

When you bind a property to another property that depends on a conditional namespace, you must use the same condition on both properties. Here, `colorPicker.Color` depends on the 'contract5Present' conditional namespace, so you must also place the 'contract5Present' prefix on the SolidColorBrush.Color property. (Or, you can place the 'contract5Present' prefix on the SolidColorBrush instead of on the Color property.) If you don't, you'll get a compile-time error.

XAML

```
<SolidColorBrush contract5Present:Color="{x:Bind colorPicker.Color,
Mode=OneWay}"/>
```

Here's the complete XAML that demonstrates these scenarios. This example contains a rectangle and a UI that lets you set the color of the rectangle.

When the app runs on the Fall Creators Update, you use a ColorPicker to let the user set the color. The ColorPicker isn't available prior to the Fall Creators Update, so when the app runs on earlier versions, you use a combo box to provide simplified color choices to the user.

XAML

```
<Page
    x:Class="ConditionalTest.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:contract5Present="http://schemas.microsoft.com/winfx/2006/xaml/present
ation?IsApiContractPresent(Windows.Foundation.UniversalApiContract,5)"

xmlns:contract5NotPresent="http://schemas.microsoft.com/winfx/2006/xaml/pres
entation?
IsApiContractNotPresent(Windows.Foundation.UniversalApiContract,5)">

    <Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
        <Grid.ColumnDefinitions>
            <ColumnDefinition/>
            <ColumnDefinition/>
        </Grid.ColumnDefinitions>

        <Rectangle x:Name="colorRectangle" Width="200" Height="200"
                   contract5NotPresent:Fill="{x:Bind ((SolidColorBrush)
((FrameworkElement)colorComboBox.SelectedItem).Tag), Mode=OneWay}">
            <contract5Present:Rectangle.Fill>
                <SolidColorBrush contract5Present:Color="{x:Bind
colorPicker.Color, Mode=OneWay}"/>
            </contract5Present:Rectangle.Fill>
        </Rectangle>

        <contract5Present:ColorPicker x:Name="colorPicker"
                                      Grid.Column="1"
                                      VerticalAlignment="Center"/>

        <contract5NotPresent:ComboBox x:Name="colorComboBox"
                                      PlaceholderText="Pick a color"
                                      Grid.Column="1"
                                      VerticalAlignment="Center">
            <ComboBoxItem>Red
                <ComboBoxItem.Tag>
                    <SolidColorBrush Color="Red"/>
                </ComboBoxItem.Tag>
            </ComboBoxItem>
            <ComboBoxItem>Blue
                <ComboBoxItem.Tag>
                    <SolidColorBrush Color="Blue"/>
                </ComboBoxItem.Tag>
            </ComboBoxItem>
            <ComboBoxItem>Green
                <ComboBoxItem.Tag>
                    <SolidColorBrush Color="Green"/>
                </ComboBoxItem.Tag>
            </ComboBoxItem>
        </contract5NotPresent:ComboBox>
    </Grid>
</Page>
```

Related articles

- Guide to UWP apps
- Dynamically detecting features with API contracts
- Programming with extension SDKs

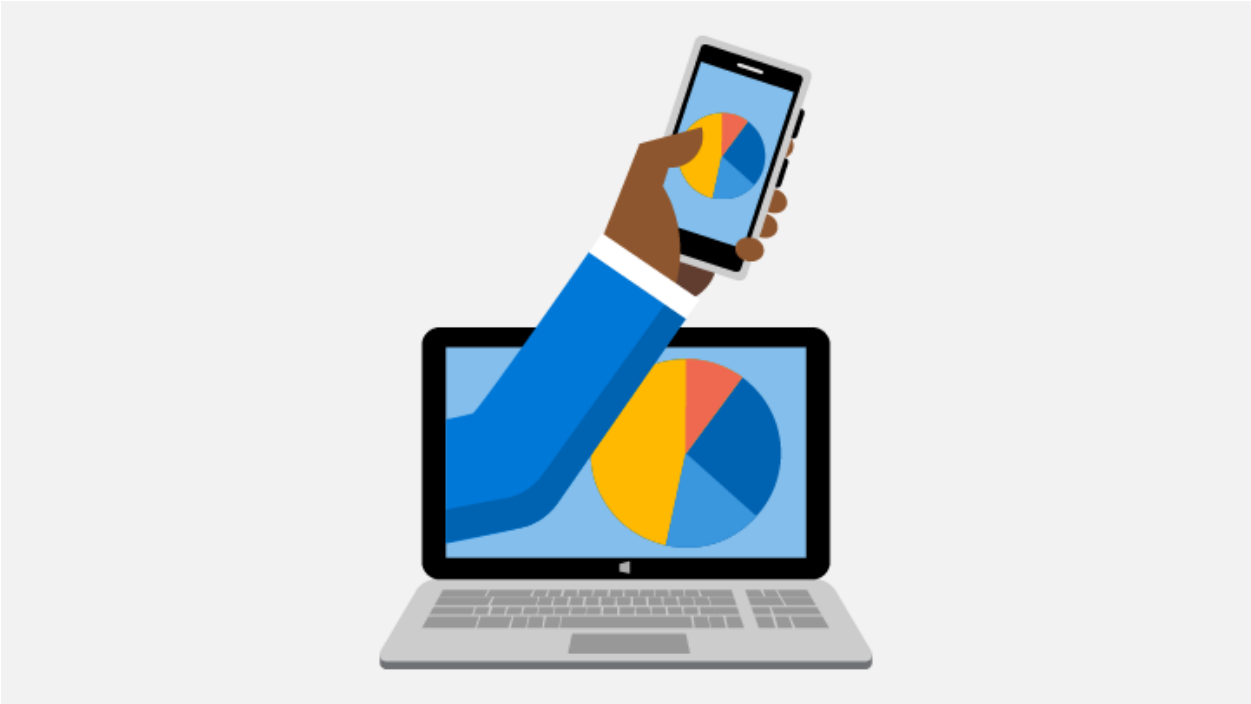
Monetization, engagement, and Store services

Article • 06/19/2023

The Windows SDK, Microsoft Advertising SDK, Microsoft Store Services SDK, and the Microsoft Store provide features that enable you to make more money from your apps and gain customers by engaging with your users. The topics in this section show you how to build these features into your app.

For details about the fees charged by the Microsoft Store and how you get paid for the money your app makes, see Getting paid.

Choose a pricing model



Charge a price for your app

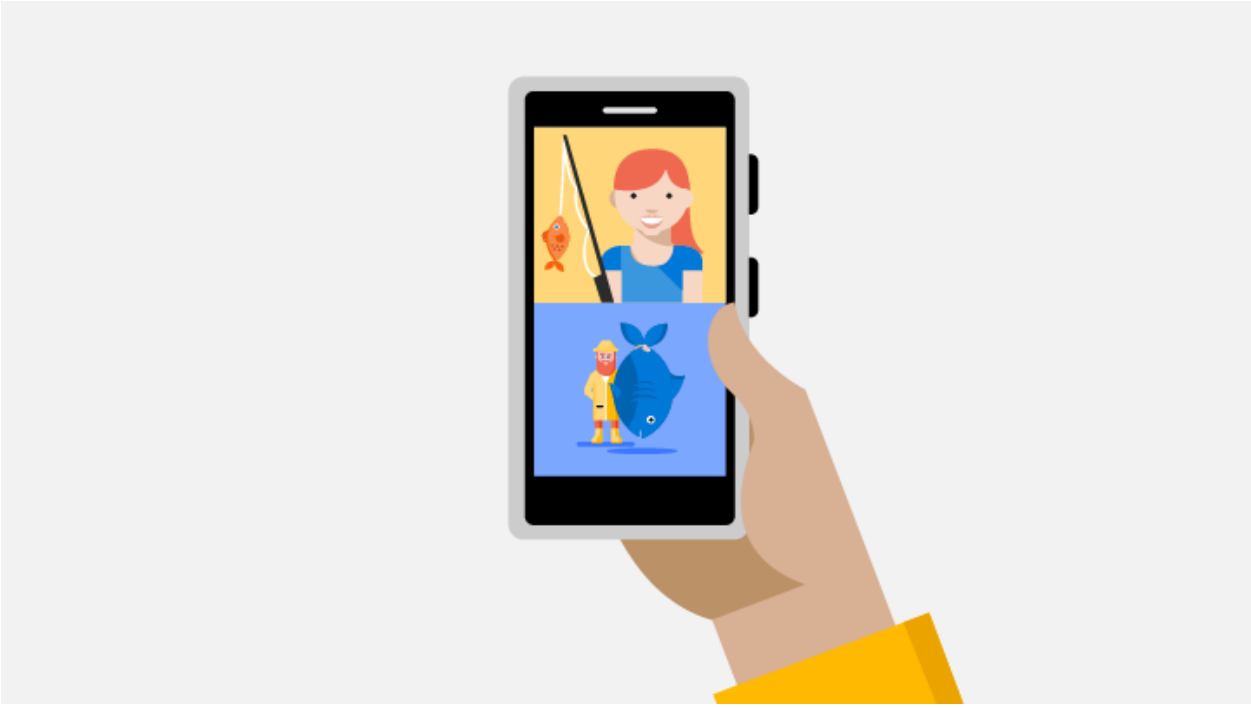
You can charge a price for your app up front. We support a comprehensive range of price tiers, including the option to set per-market prices. You can even schedule a sale to reduce the price of your app for a limited period of time.

Set app pricing

Free trials

You can offer a free trial version of your app to get more customers to try it. To entice customers to buy the full version, you can limit the features in the trial version (for example, only including the first level of a game), show ads, or specify a time-limited trial.

Offer a trial



In-app purchases

Whether you charge a price for your app or offer it for free, you can use in-app purchases in your app to provide an ongoing revenue stream. Use in-app purchases to let customers upgrade from a free to a paid version of your app, or offer durable or consumable add-ons for sale within your app.

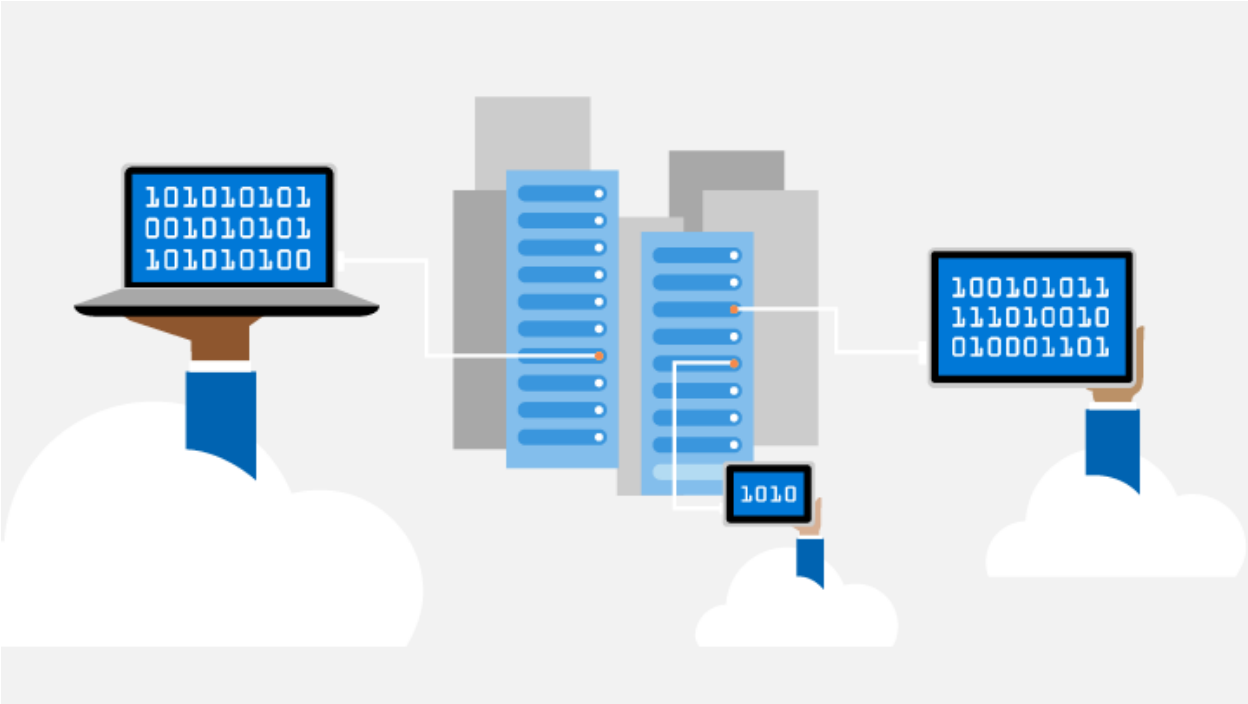
Use in-app purchases

Monetize your app with ads

Ads for every context

We support a wide variety of ad experiences to suit most needs, including banner ads, interstitial ads (banner and video), linear video ads, playable ads, and native ads. Our platform is compliant with the OpenRTB, VAST 2.x, MRAID 2, and VPAID 3 standards and is compatible with MOAT and IAS.

Explore ad options Install ad SDK



Ad mediation service

Maximize ad revenue in your apps by using the microsoft ad mediation service to serve ads from multiple popular ad networks. You can configure your mediation settings in Partner Center without touching a line of code. If you let us configure mediation for you, our machine learning algorithms will help you maximize ad revenue across the markets your app supports.

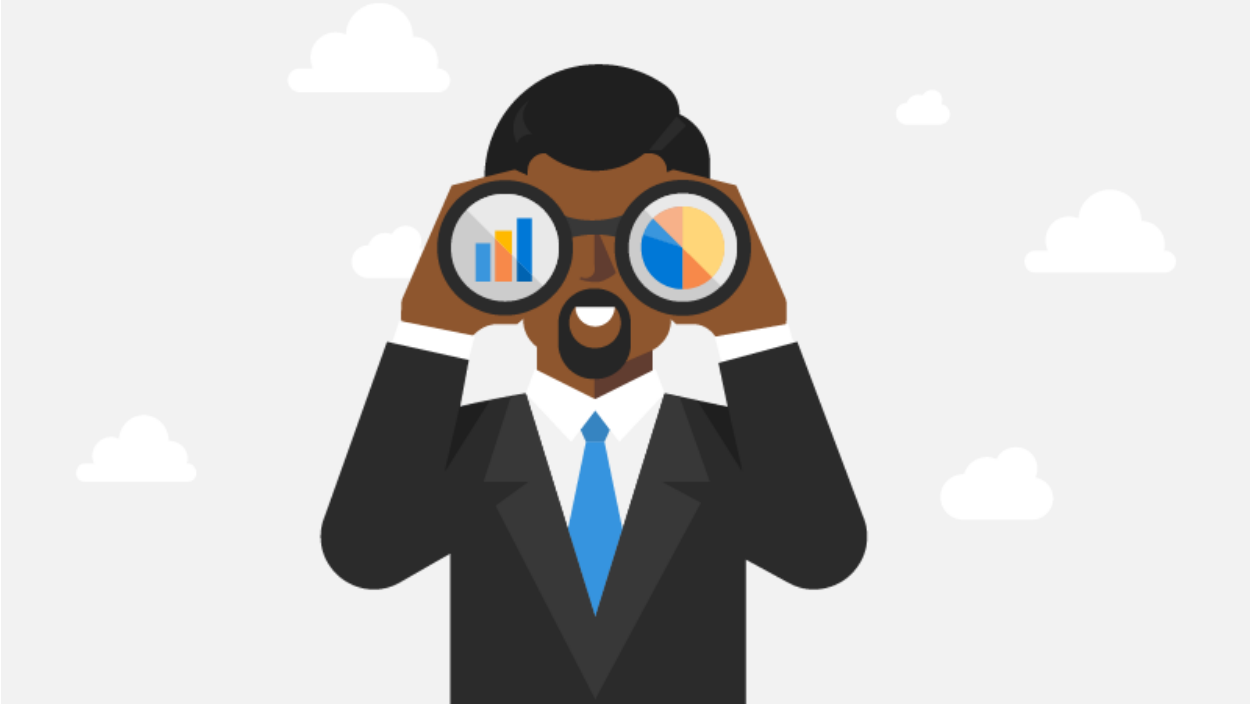
Use ad service

Analytics

Detailed analytics reports let you see how your ads in apps are performing, giving you the information you need to maximize your ad revenue. We also provide a RESTful API you can you use to get this data programmatically.

Review performance

Other monetization opportunities



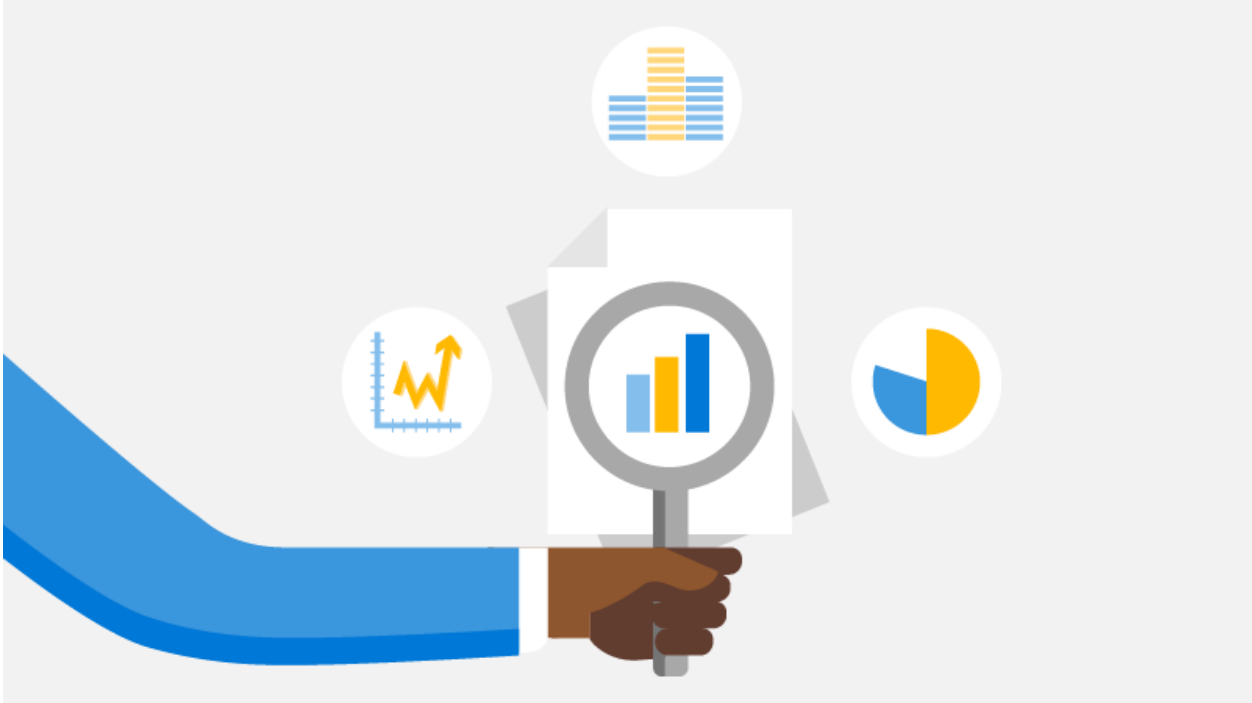
Looking for other ways to increase your monetization? Consider these options.

Expand table

Topic	Description
[Microsoft Affiliate Program]((https://www.microsoft.com/microsoft-365/business/microsoft-365-affiliate-program)	Earn commissions by linking to Microsoft products from your app, blog, web page or other communications. You can link to apps, games, music, movies, hardware, accessories, and other goods sold in the Microsoft Store.
A/B experimentation	Run A/B tests in your apps to measure the effectiveness of feature changes on some customers before you enable the changes for everyone.
Engage customers with the Microsoft Store Services SDK	The Microsoft Store Services SDK provides libraries and tools that you can use to add features to your apps that help you engage with your customers. These features include

Topic	Description
	targeted notifications, A/B tests, and launching Feedback Hub from your app.
Launch Feedback Hub from your app	Add code to your UWP apps to direct your Windows 10 and Windows 11 customers to Feedback Hub, where they can submit problems, suggestions, and upvotes. Then, manage this feedback in the Feedback report in Partner Center. This feature requires the Microsoft Store Services SDK.
Configure your app to receive Partner Center push notifications	Register a notification channel for your UWP app so it can receive Partner Center push notifications, and track the rate of app launches that resulted from the push notifications. This feature requires the Microsoft Store Services SDK.
Log custom events for Partner Center	Log custom events from your UWP app and review the events in the Usage report in Partner Center. This feature requires the Microsoft Store Services SDK.
Request ratings and reviews	Encourage your customers to rate or review your app by programmatically showing a rating and review UI.
Microsoft Store services	Learn how to use RESTful APIs to automate submissions to the Store, access analytics data for your apps, and automate other tasks related to the Store.
Add retail demo (RDX) features to your app	Include a retail demo mode in your Windows app so customers who try out PCs and devices on the sales floor can jump right in.

Monetization analytics



Keep tabs on how your app is performing in the Store by using these reports.

- Payout summary
- Acquisitions report
- Add-on acquisitions report
- Advertising performance report
- Get analytics data using our REST API
- Create customer segments
- Feedback report
- Usage report

In-app purchases and trials

Article • 01/04/2023

The Windows SDK provides APIs you can use to implement the following features to make more money from your Universal Windows Platform (UWP) app:

- **In-app purchases** Whether your app is free or not, you can sell content or new app functionality (such as unlocking the next level of a game) from right within the app.
- **Trial functionality** If you configure your app as a free trial in Partner Center, you can entice your customers to purchase the full version of your app by excluding or limiting some features during the trial period. You can also enable features, such as banners or watermarks, that are shown only during the trial, before a customer buys your app.

This article provides an overview of how in-app purchases and trials work in UWP apps.

Choose which namespace to use

There are two different namespaces you can use to add in-app purchases and trial functionality to your UWP apps, depending on which version of Windows 10 or Windows 11 your apps target. Although the APIs in these namespaces serve the same goals, they are designed quite differently, and code is not compatible between the two APIs.

- **Windows.Services.Store** Starting in Windows 10, version 1607, apps can use the API in this namespace to implement in-app purchases and trials. We recommend that you use the members in this namespace if your app project targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. This namespace supports the latest add-on types, such as Store-managed consumable add-ons, and is designed to be compatible with future types of products and features supported by Partner Center and the Store. For more information about this namespace, see the In-app purchases and trials using the Windows.Services.Store namespace section in this article.
- **Windows.ApplicationModel.Store** All versions of Windows 10 and Windows 11 also support an older API for in-app purchases and trials in this namespace. For information about the **Windows.ApplicationModel.Store** namespace, see In-app purchases and trials using the Windows.ApplicationModel.Store namespace.

> **Important**
>
> The **Windows.ApplicationModel.Store** namespace is no longer being updated with new features, and we recommend that you use the **Windows.Services.Store** namespace instead if possible for your app. The **Windows.ApplicationModel.Store** namespace is not supported in Windows desktop applications that use the Desktop Bridge or in apps or games that use a development sandbox in Partner Center (for example, this is the case for any game that integrates with Xbox Live).

Basic concepts

Every item that is offered in the Store is generally called a *product*. Most developers only work with the following types of products: *apps* and *add-ons*.

An add-on is a product or feature that you make available to your customers in the context of your app: for example, currency to be used in an app or game, new maps or weapons for a game, the ability to use your app without ads, or digital content such as music or videos for apps that have the ability to offer that type of content. Every app and add-on has an associated license that indicates whether the user is entitled to use the app or add-on. If the user is entitled to use the app or add-on as a trial, the license also provides additional info about the trial.

To offer an add-on to customers in your app, you must define the add-on for your app in Partner Center so the Store knows about it. Then, your app can use APIs in the **Windows.Services.Store** or **Windows.ApplicationModel.Store** namespace to offer the add-on for sale to the user as an in-app purchase.

UWP apps can offer the following types of add-ons.

Expand table

Add-on type	Description
Durable	An add-on that persists for the lifetime that you specify in Partner Center. By default, durable add-ons never expire, in which case they can only be purchased once. If you specify a particular duration for the add-on, the user can repurchase the add-on after it expires.
Developer-managed consumable	An add-on that can be purchased, used, and then purchased again after it is consumed. You are responsible for keeping track of the user's balance of items that the add-on represents.

Add-on type	Description
	When the user consumes any items that are associated with the add-on, you are responsible for maintaining the user's balance and for reporting the purchase of the add-on as fulfilled to the Store after the user has consumed all the items. The user cannot purchase the add-on again until your app has reported the previous add-on purchase as fulfilled. For example, if your add-on represents 100 coins in a game and the user consumes 10 coins, your app or service must maintain the new remaining balance of 90 coins for the user. After the user has consumed all 100 coins, your app must report the add-on as fulfilled, and then the user can purchase the 100 coin add-on again.
Store-managed consumable	An add-on that can be purchased, used, and purchased again at any time. The Store keeps track of the user's balance of items that the add-on represents. When the user consumes any items that are associated with the add-on, you are responsible for reporting those items as fulfilled to the Store, and the Store updates the user's balance. The user can purchase the add-on as many times as they want (they do not need to consume the items first). Your app can query for the current balance for the user at any time. For example, if your add-on represents an initial quantity of 100 coins in a game and the user consumes 50 coins, your app reports to the Store that 50 units of the add-on were fulfilled, and the Store updates the remaining balance. If the user then repurchases your add-on to acquire 100 more coins, they will now have 150 coins total. **Note** To use Store-managed consumables, your app must target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio, and it must use the **Windows.Services.Store** namespace instead of the **Windows.ApplicationModel.Store** namespace.
Subscription	A durable add-on where the customer continues to be charged at recurring intervals in order to keep using the add-on. The customer can cancel the subscription at any time to avoid further charges. **Note** To use subscription add-ons, your app must target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio, and it must use the **Windows.Services.Store** namespace instead of the **Windows.ApplicationModel.Store** namespace.

> **Note**
>
> Other types of add-ons, such as durable add-ons with packages (also known as downloadable content or DLC) are only available to a restricted set of developers, and are not covered in this documentation.

In-app purchases and trials using the Windows.Services.Store namespace

This section provides an overview of important tasks and concepts for the Windows.Services.Store namespace. This namespace is available only to apps that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio (this corresponds to Windows 10, version 1607). We recommend that apps use the **Windows.Services.Store** namespace instead of the Windows.ApplicationModel.Store namespace if possible. For information about the **Windows.ApplicationModel.Store** namespace, see this article.

In this section

- Video
- Get started with the StoreContext class
- Implement in-app purchases
- Implement trial functionality
- Test your in-app purchase or trial implementation
- Receipts for in-app purchases
- Using the StoreContext class with the Desktop Bridge
- Products, SKUs, and availabilities
- Store IDs

Get started with the StoreContext class

The main entry point to the **Windows.Services.Store** namespace is the StoreContext class. This class provides methods you can use to get info for the current app and its available add-ons, get license info for the current app or its add-ons, purchase an app or add-on for the current user, and perform other tasks. To get a StoreContext object, do one of the following:

- In a single-user app (that is, an app that runs only in the context of the user that launched the app), use the static GetDefault method to get a **StoreContext** object that you can use to access Microsoft Store-related data for the user. Most Universal Windows Platform (UWP) apps are single-user apps.

  C#

  ```
  Windows.Services.Store.StoreContext context =
  StoreContext.GetDefault();
  ```

- In a multi-user app, use the static GetForUser method to get a **StoreContext** object that you can use to access Microsoft Store-related data for a specific user who is signed in with their Microsoft account while using the app. The following example gets a **StoreContext** object for the first available user.

```
C#

var users = await Windows.System.User.FindAllAsync();
Windows.Services.Store.StoreContext context =
StoreContext.GetForUser(users[0]);
```

> **Note**
>
> Windows desktop applications that use the Desktop Bridge must perform additional steps to configure the StoreContext object before they can use this object. For more information, see this section.

After you have a StoreContext object, you can start calling methods of this object to get Store product info for the current app and its add-ons, retrieve license info for the current app and its add-ons, purchase an app or add-on for the current user, and perform other tasks. For more information about common tasks you can perform using this object, see the following articles:

- Get product info for apps and add-ons
- Get license info for apps and add-ons
- Enable in-app purchases of apps and add-ons
- Enable consumable add-on purchases
- Enable subscription add-ons for your app
- Implement a trial version of your app

For a sample app that demonstrates how to use **StoreContext** and other types in the **Windows.Services.Store** namespace, see the Store sample.

Implement in-app purchases

To offer an in-app purchase to customers in your app using the **Windows.Services.Store** namespace:

1. If your app offers add-ons that customers can purchase, create add-on submissions for your app in Partner Center .

2. Write code in your app to retrieve product info for your app or an add-on offered by your app and then determine whether the license is active (that is, whether the user has a license to use the app or add-on). If the license isn't active, display a UI that offers the app or add-on for sale to the user as an in-app purchase.

3. If the user chooses to purchase your app or add-on, use the appropriate method to purchase the product:

    - If the user is purchasing your app or a durable add-on, follow the instructions in Enable in-app purchases of apps and add-ons.
    - If the user is purchasing a consumable add-on, follow the instructions in Enable consumable add-on purchases.
    - If the user is purchasing a subscription add-on, follow the instructions in Enable subscription add-ons for your app.

4. Test your implementation by following the testing guidance in this article.

Implement trial functionality

To exclude or limit features in a trial version of your app using the **Windows.Services.Store** namespace:

1. Configure your app as a free trial in Partner Center.

2. Write code in your app to retrieve product info for your app or an add-on offered by your app and then determine whether the license associated with the app is a trial license.

3. Exclude or limit certain features in your app if it is a trial, and then enable the features when the user purchases a full license. For more information and a code example, see Implement a trial version of your app.

4. Test your implementation by following the testing guidance in this article.

Test your in-app purchase or trial implementation

If your app uses APIs in the **Windows.Services.Store** namespace to implement in-app purchase or trial functionality, you must publish your app to the Store and download the app to your development device to use its license for testing. Follow this process to test your code:

1. If your app is not yet published and available in the Store, make sure your app meets the minimum Windows App Certification Kit requirements, submit your app

in Partner Center, and make sure your app passes the certification process. You can configure your app so it is not discoverable in the Store while you test it. Please note the proper configuration of package flights. Incorrectly configured package flights may be not be able to be downloaded.

2. Next, make sure you have completed the following:

   - Write code in your app that uses the StoreContext class and other related types in the **Windows.Services.Store** namespace to implement in-app purchases or trial functionality.
   - If your app offers an add-on that customers can purchase, create an add-on submission for your app in Partner Center.
   - If you want to exclude or limit some features in a trial version of your app, configure your app as a free trial in Partner Center.

3. With your project open in Visual Studio, click the **Project menu**, point to **Store**, and then click **Associate App with the Store**. Complete the instructions in the wizard to associate the app project with the app in your Partner Center account that you want to use for testing.

   > ⓘ **Note**
   >
   > If you do not associate your project with an app in the Store, the StoreContext methods set the **ExtendedError** property of their return values to the error code value 0x803F6107. This value indicates that the Store doesn't have any knowledge about the app.

4. If you have not done so already, install the app from the Store that you specified in the previous step, run the app once, and then close this app. This ensures that a valid license for the app is installed to your development device.

5. In Visual Studio, start running or debugging your project. Your code should retrieve app and add-on data from the Store app that you associated with your local project. If you are prompted to reinstall the app, follow the instructions and then run or debug your project.

   > ⓘ **Note**
   >
   > After you complete these steps, you can continue to update your app's code and then debug your updated project on your development computer without submitting new app packages to the Store. You only need to download the Store version of your app to your development computer once

to obtain the local license that will be used for testing. You only need to submit new app packages to the Store after you complete your testing and you want to make the in-app purchase or trial-related features in your app available to your customers.

If your app uses the **Windows.ApplicationModel.Store** namespace, you can use the CurrentAppSimulator class in your app to simulate license info during testing before you submit your app to the Store. For more information, see Get started with the CurrentApp and CurrentAppSimulator classes.

> **Note**
>
> The **Windows.Services.Store** namespace does not provide a class that you can use to simulate license info during testing. If you use the **Windows.Services.Store** namespace to implement in-app purchases or trials, you must publish your app to the Store and download the app to your development device to use its license for testing as described above.

Receipts for in-app purchases

The **Windows.Services.Store** namespace does not provide an API you can use to obtain a transaction receipt for successful purchases in your app's code. This is a different experience from apps that use the **Windows.ApplicationModel.Store** namespace, which can use a client-side API to retrieve a transaction receipt.

If you implement in-app purchases using the **Windows.Services.Store** namespace and you want to validate whether a given customer has purchased an app or add-on, you can use the query for products method in the Microsoft Store collection REST API. The return data for this method confirms whether the specified customer has an entitlement for a given product, and provides data for the transaction in which the user acquired the product. The Microsoft Store collection API uses Azure AD authentication to retrieve this information.

Using the StoreContext class with the Desktop Bridge

Desktop applications that use the Desktop Bridge can use the StoreContext class to implement in-app purchases and trials. However, if you have a Win32 desktop application or a desktop application that has a window handle (HWND) that is associated with the rendering framework (such as a WPF or Windows App SDK application), your application must configure the **StoreContext** object to specify which

application window is the owner window for modal dialogs that are shown by the object.

Many **StoreContext** members (and members of other related types that are accessed through the **StoreContext** object) display a modal dialog to the user for Store-related operations such as purchasing a product. If a desktop application does not configure the **StoreContext** object to specify the owner window for modal dialogs, this object will return inaccurate data or errors.

To configure a **StoreContext** object in a desktop application that uses the Desktop Bridge, follow these steps.

For .NET 6 or later

If your application is written in C# with .NET 6 or later, then follow these steps.

1. Make sure that the `TargetFramework` property in the project file is set to a specific Windows SDK version to access the Windows Runtime APIs, which provides access to the **WinRT.Interop** namespace. For example:

   XML

   ```
   <PropertyGroup>
     <!-- You can also target other versions of the Windows SDK and .NET,
   e.g. "net6.0-windows10.0.19041.0" -->
     <TargetFramework>net6.0-windows10.0.22000.0</TargetFramework>
   </PropertyGroup>
   ```

2. Get a StoreContext object by using the GetDefault method (or GetForUser if your app is a multi-user app) as described earlier in this article). To initialize the dialog with the specified window handle, use the **WinRT.Interop.WindowNative.GetWindowHandle** and **WinRT.Interop.InitializeWithWindow.Initialize** methods (see Retrieve a window handle (HWND) and Display WinRT UI objects that depend on CoreWindow).

   C#

   ```
   StoreContext context = StoreContext.GetDefault();
   // Obtain window handle by passing in pointer to the window object
   var hwnd = WinRT.Interop.WindowNative.GetWindowHandle(windowObject);
   // Initialize the dialog using wrapper function for
   IInitializeWithWindow
   WinRT.Interop.InitializeWithWindow.Initialize(context, hwnd);
   ```

For earlier versions of .NET or C++

If your application is written with an earlier version of .NET or in C++, follow these steps.

1. Do one of the following to enable your app to access the IInitializeWithWindow interface:

    - If your application is written in a managed language such as C# or Visual Basic (prior to .NET 6), then declare the **IInitializeWithWindow** interface in your app's code with the ComImport attribute as shown in the following C# example. This example assumes that your code file has a **using** statement for the **System.Runtime.InteropServices** namespace.

      C#

      ```
      [ComImport]
      [Guid("3E68D4BD-7135-4D10-8018-9FB6D9F33FA1")]
      [InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
      public interface IInitializeWithWindow
      {
          void Initialize(IntPtr hwnd);
      }
      ```

    - If your application is written in C++, then add a reference to the `shobjidl.h` header file in your code. This header file contains the declaration of the **IInitializeWithWindow** interface.

2. Get a StoreContext object by using the GetDefault method (or GetForUser if your app is a multi-user app) as described earlier in this article, and cast this object to an IInitializeWithWindow object. Then, call the IInitializeWithWindow.Initialize method, and pass the handle of the window that you want to be the owner for any modal dialogs that are shown by **StoreContext** methods. The following C# example shows how to pass the handle of your app's main window to the method. Also see Retrieve a window handle (HWND) and Display WinRT UI objects that depend on CoreWindow.

   C#

   ```
   StoreContext context = StoreContext.GetDefault();
   IInitializeWithWindow initWindow = (IInitializeWithWindow)
   (object)context;
   initWindow.Initialize(System.Diagnostics.Process.GetCurrentProcess().Ma
   inWindowHandle);
   ```

Products, SKUs, and availabilities

Every product in the Store has at least one *SKU*, and each SKU has at least one *availability*. These concepts are abstracted away from most developers in Partner Center, and most developers will never define SKUs or availabilities for their apps or add-ons. However, because the object model for Store products in the **Windows.Services.Store** namespace includes SKUs and availabilities, a basic understanding of these concepts can be helpful for some scenarios.

Expand table

Object	Description
Product	A *product* refers to any type of product that is available in the Store, including an app or add-on. Each product in the Store has a corresponding StoreProduct object. This class provides properties you can use to access data such as the Store ID of the product, the images and videos for the Store listing, and pricing info. It also provides methods you can use to purchase the product.
SKU	A *SKU* is a specific version of a product with its own description, price, and other unique product details. Each app or add-on has a default SKU. The only time most developers will ever have multiple SKUs for an app is if they publish a full version of their app and a trial version (in the Store catalog, each of these versions is a different SKU of the same app). Some publishers have the ability to define their own SKUs. For example, a large game publisher might release a game with one SKU that shows green blood in markets that don't allow red blood and a different SKU that shows red blood in all other markets. Alternatively, a publisher who sells digital video content might publish two SKUs for a video, one SKU for the high-definition version and a different SKU for the standard-definition version. Each SKU in the Store has a corresponding StoreSku object. Every StoreProduct has a Skus property you can use to access the SKUs for the product.
Availability	An *availability* is a specific version of a SKU with its own unique pricing info. Each SKU has a default availability. Some publishers have the ability to define their own availabilities to introduce different price options for a given SKU. Each availability in the Store has a corresponding StoreAvailability object. Every StoreSku has an Availabilities property you can use to access the availabilities for the SKU. For most developers, each SKU has a single default availability.

Store IDs

Every app, add-on, or other product in the Store has an associated **Store ID** (this is also sometimes called a *product Store ID*). Many APIs require the Store ID in order to perform

an operation on an app or add-on.

The Store ID of any product in the Store is 12-character alpha-numeric string, such as `9NBLGGH4R315`. There are several different ways to get the Store ID for a product in the Store:

- For an app, you can get the Store ID on the App identity page in Partner Center.
- For an add-on, you can get the Store ID on the add-on's overview page in Partner Center.
- For any product, you can also get the Store ID programmatically by using the StoreId property of the StoreProduct object that represents the product.

For products with SKUs and availabilities, the SKUs and availabilities also have their own Store IDs with different formats.

Expand table

Object	Store ID format
SKU	The Store ID for a SKU has the format `<product Store ID>/xxxx`, where `xxxx` is a 4-character alpha-numeric string that identifies a SKU for the product. For example, `9NBLGGH4R315/000N`. This ID is returned by the StoreId property of a StoreSku object, and it is sometimes called the *SKU Store ID*.
Availability	The Store ID for an availability has the format `<product Store ID>/xxxx/yyyyyyyyyyyy`, where `xxxx` is a 4-character alpha-numeric string that identifies a SKU for the product and `yyyyyyyyyyyy` is a 12-character alpha-numeric string that identifies an availability for the SKU. For example, `9NBLGGH4R315/000N/4KW6QZD2VN6X`. This ID is returned by the StoreId property of a StoreAvailability object, and it is sometimes called the *availability Store ID*.

How to use product IDs for add-ons in your code

If you want to make an add-on available to your customers in the context of your app, you must enter a unique product ID for your add-on when you create your add-on submission in Partner Center. You can use this product ID to refer to the add-on in your code, although the specific scenarios in which you can use the product ID depend on which namespace you use for in-app purchases in your app.

Note

The product ID that you enter in Partner Center for an add-on is different than the add-on's Store ID. The Store ID is generated by Partner Center.

Apps that use the Windows.Services.Store namespace

If your app uses the **Windows.Services.Store** namespace, you can use the product ID to easily identify the StoreProduct that represents your add-on or the StoreLicense that represents your add-on's license. The product ID is exposed by the StoreProduct.InAppOfferToken and StoreLicense.InAppOfferToken properties.

> **Note**
>
> Although the product ID is a useful way to identify an add-on in your code, most operations in the **Windows.Services.Store** namespace use the Store ID of an add-on instead of the product ID. For example, to programmatically retrieve one or more known add-ons for an app, pass the Store IDs (rather than the product IDs) of the add-ons to the GetStoreProductsAsync method. Similarly, to report a consumable add-on as fulfilled, pass the Store ID of the add-on (rather than the product ID) to the ReportConsumableFulfillmentAsync method.

Apps that use the Windows.ApplicationModel.Store namespace

If your app uses the **Windows.ApplicationModel.Store** namespace, you'll need to use the product ID that you assign to an add-on in Partner Center for most operations. For example:

- Use the product ID to identify the ProductListing that represents your add-on or the ProductLicense that represents your add-on's license. The product ID is exposed by the ProductListing.ProductId and ProductLicense.ProductId properties.
- Specify the product ID when you purchase your add-on for the user by using the RequestProductPurchaseAsync method. For more information, see Enable in-app product purchases.
- Specify the product ID when you report your consumable add-on as fulfilled by using the ReportConsumableFulfillmentAsync method. For more information, see Enable consumable in-app product purchases.

Related topics

- Get product info for apps and add-ons
- Get license info for apps and add-ons
- Enable in-app purchases of apps and add-ons
- Enable consumable add-on purchases
- Enable subscription add-ons for your app
- Implement a trial version of your app
- Error codes for Store operations
- In-app purchases and trials using the Windows.ApplicationModel.Store namespace

---

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Get product info for apps and add-ons

Article • 10/24/2022

This article demonstrates how to use methods of the StoreContext class in the Windows.Services.Store namespace to access Store-related info for the current app or one of its add-ons.

For a complete sample application, see the Store sample.

> **Note**
>
> The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. If your app targets an earlier version of Windows 10, you must use the **Windows.ApplicationModel.Store** namespace instead of the **Windows.Services.Store** namespace. For more information, see **this article**.

Prerequisites

These examples have the following prerequisites:

- A Visual Studio project for a Universal Windows Platform (UWP) app that targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release.
- You have created an app submission in Partner Center and this app is published in the Store. You can optionally configure the app so it is not discoverable in the Store while you test it. For more information, see our testing guidance.
- If you want to get product info for an add-on for the app, you must also create the add-on in Partner Center.

The code in these examples assume:

- The code runs in the context of a Page that contains a ProgressRing named `workingProgressRing` and a TextBlock named `textBlock`. These objects are used to indicate that an asynchronous operation is occurring and to display output messages, respectively.
- The code file has a **using** statement for the **Windows.Services.Store** namespace.
- The app is a single-user app that runs only in the context of the user that launched the app. For more information, see In-app purchases and trials.